

Bash Scripting, Task Automation & Scheduling (Cron Jobs)

Author:

Sivateja Mattaparthi

Role Focus

IT Support • Help Desk • Service Desk • Desktop Support

Lab Objectives

- Create and execute Bash scripts.
- Work with variables and user input.
- Implement conditional statements and loops.
- Automate repetitive file management tasks.
- Configure scheduled task execution using Cron.
- Verify successful script execution and scheduling.

Component	Details
Operating System	macOS Catalina
Environment	Home Lab
Shell	Bash
Scheduler	Cron
Tools Used	Terminal, Bash, Cron
Platform	Physical MacBook Air 2019

Introduction

Bash scripting is a fundamental skill for IT Support, System Administrators, Cloud Engineers, and DevOps professionals. It enables administrators to automate repetitive tasks, manage files, monitor system resources, and schedule routine maintenance activities. This hands-on lab demonstrates the basics of Bash scripting, including variables, user input, conditional statements, loops, file automation, and task scheduling using Cron.

Creating and Executing a System Information Bash Script

Created a Bash script (sys_info.sh) to automate the collection of basic system information. The script gathers details such as the hostname, current user, working directory, date and time, disk usage, memory usage, and system uptime. After creating the script, execution permissions were assigned using `chmod +x`, and the script was successfully executed from the Terminal to generate a system information report.

Commands Used

- `mkdir` – Create the project directory.
- `touch` – Create a new Bash script.
- `nano` – Edit the script.
- `cat` – Display the script contents.
- `chmod +x` – Make the script executable.
- `./sys_info.sh` – Execute the Bash script.
- `hostname` – Display the system hostname.
- `whoami` – Display the logged-in user.
- `pwd` – Show the current working directory.
- `date` – Display the current system date and time.
- `df -h` – Display disk usage in a human-readable format.
- `uptime` – Display system uptime and load averages.

```
Last login: Wed Aug 5 20:37:11 on ttys000
sivatejamattaparathi@Sivatejas-MacBook-Air ~ % cd Desktop
sivatejamattaparathi@Sivatejas-MacBook-Air Desktop % mkdir bash_lab
sivatejamattaparathi@Sivatejas-MacBook-Air Desktop % cd bash_lab
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % touch sys_info.sh
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % nano sys_info.sh
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % cat sys_info.sh
#!/bin/bash

echo "=====
echo "System Information Report"
echo "=====

echo ""
echo "Hostname:"
hostname

echo ""
echo "Current User:"
whoami

echo ""
echo "Current Directory:"
pwd

echo ""
echo "Date & Time:"
date

echo ""
echo "Disk Usage:"
df -h

echo ""
echo "Memory Usage:"
free -h

echo ""
echo "System Uptime:"
uptime
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % chmod +x sys_info.sh
```

```
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % ./sys_info.sh
=====
System Information Report
=====

Hostname:
Sivatejas-MacBook-Air.local

Current User:
sivatejamattaparathi

Current Directory:
/Users/sivatejamattaparathi/Desktop/bash_lab

Date & Time:
Wed Aug 5 21:48:37 IST 2026

Disk Usage:


| Filesystem    | Size  | Used  | Avail | Capacity | Used   | ifree      | %used | Mounted on                |
|---------------|-------|-------|-------|----------|--------|------------|-------|---------------------------|
| /dev/disk1s5  | 113Gi | 190Gi | 60Gi  | 15%      | 488417 | 1181338023 | 0%    | /                         |
| devfs         | 193Ki | 193Ki | 0Bi   | 100%     | 676    | 0          | 100%  | /dev                      |
| /dev/disk1s1  | 113Gi | 38Gi  | 60Gi  | 3%       | 337479 | 1181480961 | 0%    | /System/Volumes/Data      |
| /dev/disk1s4  | 113Gi | 3.0Gi | 60Gi  | 5%       | 3      | 1181818437 | 0%    | /private/var/vm           |
| map auto_home | 0Bi   | 0Bi   | 0Bi   | 100%     | 0      | 0          | 100%  | /System/Volumes/Data/home |
| /dev/disk1s3  | 113Gi | 608Mi | 60Gi  | 1%       | 60     | 1181818390 | 0%    | /Volumes/Recovery         |



Memory Usage:
./sys_info.sh: line 29: free: command not found

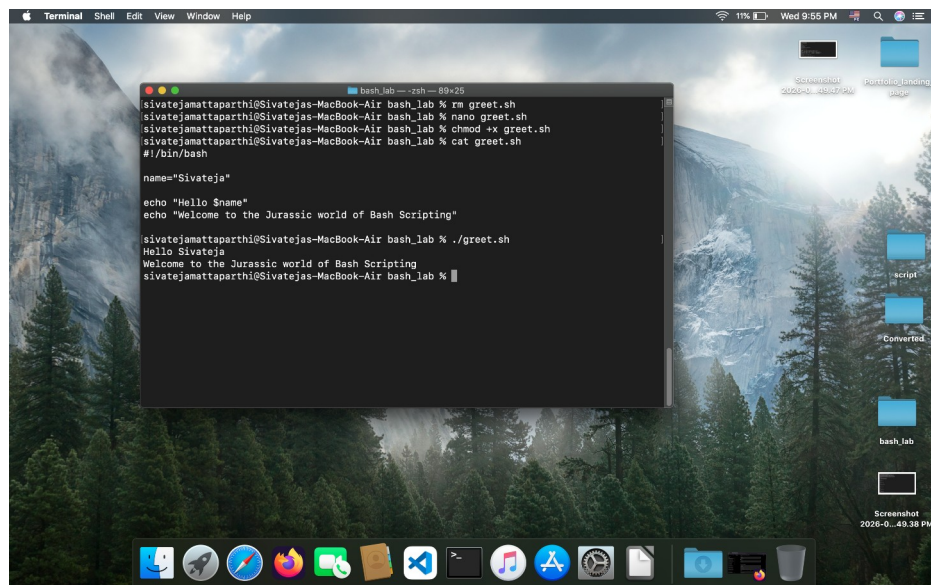
System Uptime:
21:48 up 4 day, 9:43, 2 users, load averages: 1.56 2.81 2.84
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab %
```

Working with Variables in Bash

Created a simple Bash script that stores a value in a variable and displays it using the echo command. The script was made executable and successfully executed from the Terminal, demonstrating how variables can be used to store and retrieve data within a Bash script.

Commands Used

- nano – Create and edit the Bash script.
- cat – Display the script contents.
- chmod +x – Make the script executable.
- ./greet.sh – Execute the Bash script.
- echo – Display text and variable values.



```
bash_lab --- zsh --- 89x25
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % rm greet.sh
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % nano greet.sh
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % chmod +x greet.sh
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % cat greet.sh
#!/bin/bash

name="Sivateja"

echo "Hello $name"
echo "Welcome to the Jurassic world of Bash Scripting"

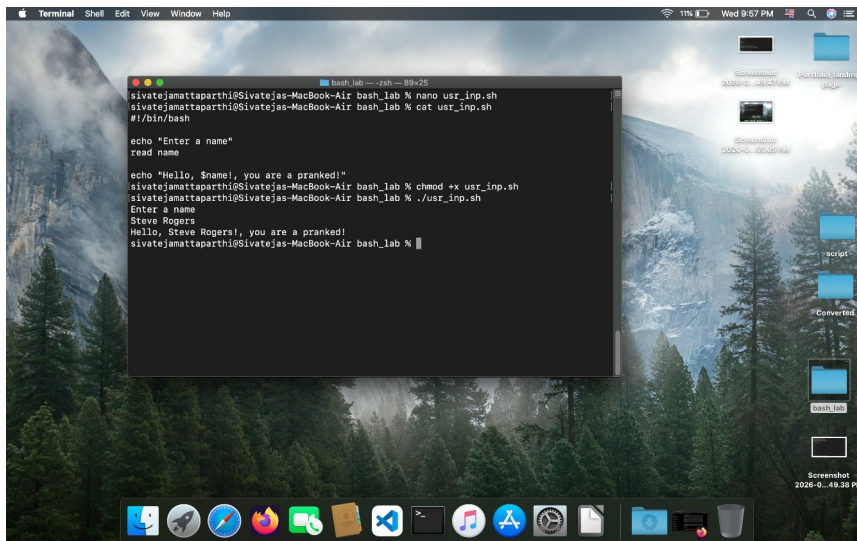
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab % ./greet.sh
Hello Sivateja
Welcome to the Jurassic world of Bash Scripting
sivatejamattaparathi@Sivatejas-MacBook-Air bash_lab %
```

Accepting User Input in Bash

Created a Bash script that prompts the user to enter a name using the read command. The entered value is stored in a variable and displayed back to the user, demonstrating how Bash scripts can interact with users during execution.

Commands Used

- nano – Create and edit the Bash script.
- cat – Display the script contents.
- chmod +x – Make the script executable.
- ./usr_inp.sh – Execute the Bash script.
- read – Accept user input.
- echo – Display the entered value.

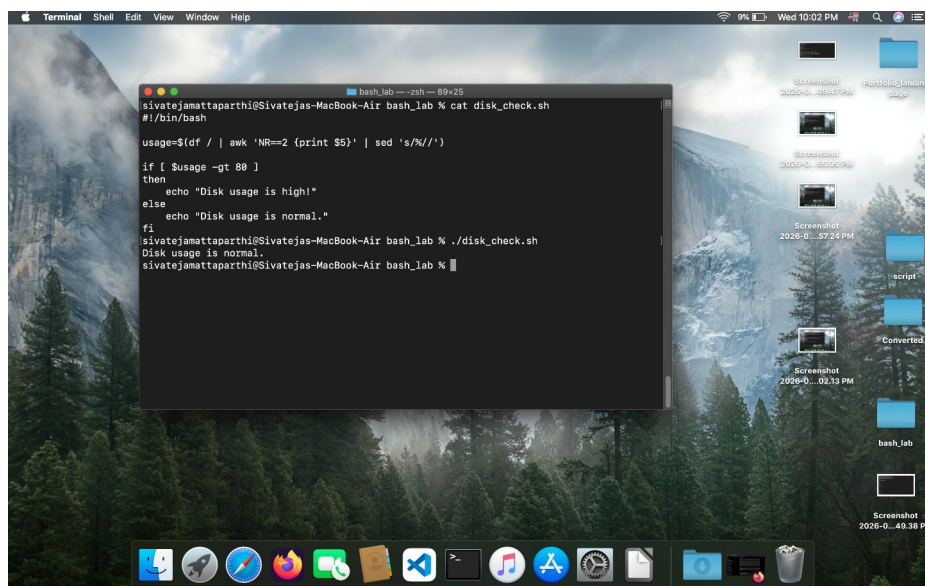


Monitoring Disk Usage with Bash

Created a Bash script that checks the current disk usage of the root filesystem using the `df` command. The script extracts the disk usage percentage with `awk` and `sed`, then uses an `if-else` statement to determine whether disk utilisation has exceeded a predefined threshold (80%).

Commands Used

- `df` – Display disk space usage.
- `awk` – Extract the disk usage percentage.
- `sed` – Remove the `%` symbol for comparison.
- `if, else` – Perform conditional checks.
- `chmod +x` – Make the script executable.
- `./disk_check.sh` – Execute the script.

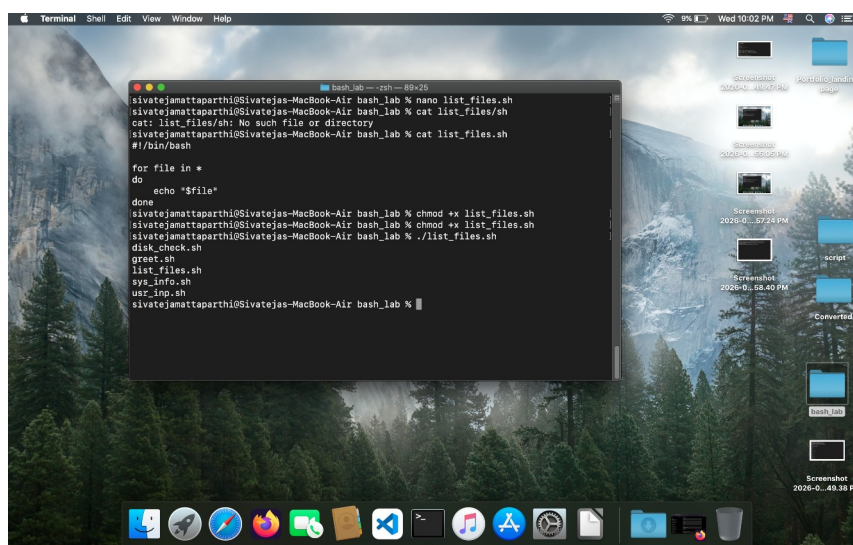


Listing Files Using a Bash Loop

Developed a Bash script that uses a for loop to iterate through all files in the current working directory. Each filename is displayed individually using the echo command, demonstrating how Bash can automate repetitive tasks through loops.

Commands Used

- nano – Create and edit the Bash script.
- cat – Display the script contents.
- chmod +x – Make the script executable.
- ./list_files.sh – Execute the script.
- for loop – Iterate through files.
- echo – Display filenames.



```
Terminal Shell Edit View Window Help
sivatejamattaparthi@Sivatejas-MacBook-Air bash_lab % nano list_files.sh
sivatejamattaparthi@Sivatejas-MacBook-Air bash_lab % cat list_files.sh
cat: list_files/sh: No such file or directory
sivatejamattaparthi@Sivatejas-MacBook-Air bash_lab % cat list_files.sh
#!/bin/bash

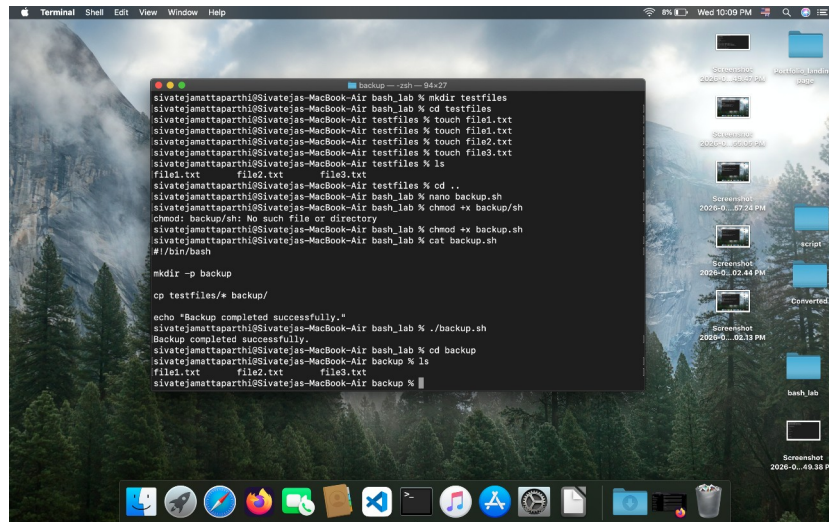
for file in *
do
    echo "$file"
done
sivatejamattaparthi@Sivatejas-MacBook-Air bash_lab % chmod +x list_files.sh
sivatejamattaparthi@Sivatejas-MacBook-Air bash_lab % ./list_files.sh
disk_check.sh
greet.sh
list_files.sh
sys_info.sh
usr_inp.sh
sivatejamattaparthi@Sivatejas-MacBook-Air bash_lab %
```

Automating File Backups with Bash

Created a Bash script to automate the backup of files from a source directory into a dedicated backup folder. The script first ensures that the backup directory exists using mkdir -p, then copies all files from the source directory using the cp command and confirms successful completion with a status message.

Commands Used

- mkdir -p – Create the backup directory if it does not already exist.
- touch – Create sample files for testing.
- cp – Copy files to the backup directory.
- ls – Verify the copied files.
- chmod +x – Make the script executable.
- ./backup.sh – Execute the backup script.



Scheduling Automated Backups with Cron

Configured a scheduled task using Cron to automate the execution of the backup script at one-minute intervals. The backup script was granted execute permissions, added to the user's crontab, and verified to ensure the scheduled job was correctly registered. The backup log file and running Cron service were also checked to confirm that the script executed successfully when run manually.

Commands Used

- `crontab -e` – Create or edit scheduled Cron jobs.
- `crontab -l` – Display configured Cron jobs.
- `chmod +x` – Grant execute permission to the backup script.
- `cat` – View the backup log.
- `ls -l` – Verify backup files and log creation.
- `ps aux | grep cron` – Confirm the Cron daemon is running.

```
sivatejamattaparthi@Sivatejas-MacBook-Air backup % pwd
/Users/sivatejamattaparthi/Desktop/bash_lab/backup
sivatejamattaparthi@Sivatejas-MacBook-Air backup % crontab -e
crontab: no crontab for sivatejamattaparthi - using an empty one
crontab: installing new crontab
sivatejamattaparthi@Sivatejas-MacBook-Air backup % chmod +x /Users/sivatejamattaparthi/Desktop/bash_lab/backup.sh
sivatejamattaparthi@Sivatejas-MacBook-Air backup % crontab -l
#1 * * * * /Users/sivatejamattaparthi/Desktop/bash_lab/backup.sh
sivatejamattaparthi@Sivatejas-MacBook-Air backup % nano /Users/sivatejamattaparthi/Desktop/bash_lab/backup.sh
sivatejamattaparthi@Sivatejas-MacBook-Air backup % cat /Users/sivatejamattaparthi/Desktop/bash_lab/backup.log
cat: /Users/sivatejamattaparthi/Desktop/bash_lab/backup.log: No such file or directory
sivatejamattaparthi@Sivatejas-MacBook-Air backup % cat /Users/sivatejamattaparthi/Desktop/bash_lab/backup.sh
#!/bin/bash

mkdir -p /Users/sivatejamattaparthi/Desktop/bash_lab/backup

cp /Users/sivatejamattaparthi/Desktop/bash_lab/testfiles/* \
/Users/sivatejamattaparthi/Desktop/bash_lab/backup/

echo "Backup completed at $(date)" >> /Users/sivatejamattaparthi/Desktop/bash_lab/backup.log
sivatejamattaparthi@Sivatejas-MacBook-Air backup % ls -l /Users/sivatejamattaparthi/Desktop/bash_lab
total 56
drwxr-xr-x  5 sivatejamattaparthi  staff  160 Aug 5 22:09 backup
-rw-r--r--  1 sivatejamattaparthi  staff   49 Aug 5 22:26 backup.log
-rwxr-xr-x  1 sivatejamattaparthi  staff  281 Aug 5 22:18 backup.sh
-rwxr-xr-x  1 sivatejamattaparthi  staff  165 Aug 5 21:57 disk_check.sh
-rwxr-xr-x  1 sivatejamattaparthi  staff  105 Aug 5 21:54 greet.sh
-rwxr-xr-x  1 sivatejamattaparthi  staff   52 Aug 5 22:01 list_files.sh
-rwxr-xr-x  1 sivatejamattaparthi  staff   371 Aug 5 21:47 sys_info.sh
drwxr-xr-x  5 sivatejamattaparthi  staff  160 Aug 5 22:08 testfiles
-rwxr-xr-x  1 sivatejamattaparthi  staff   85 Aug 5 21:56 usr_inp.sh
sivatejamattaparthi@Sivatejas-MacBook-Air backup % cat /Users/sivatejamattaparthi/Desktop/bash_lab/backup.log
Backup completed at Wed Aug 5 22:26:28 IST 2026
sivatejamattaparthi@Sivatejas-MacBook-Air backup % ps aux | grep cron
sivatejamattaparthi 36344  0.0  0.0  4268896   284 s000  R+   10:26PM   0:00.00 grep cron
root               36229  0.0  0.0  4288132   928  ??  Ss   10:15PM   0:00.02 /usr/sbin/cron
```

```

sivatejamattaparathi@Sivatejas-MacBook-Air backup % cat /Users/sivatejamattaparathi/Desktop/bash_lab/backup.log
Backup completed at Wed Aug 5 22:26:28 IST 2026
sivatejamattaparathi@Sivatejas-MacBook-Air backup % log show --last 5m --predicate 'process == "cron"'
Filtering the log data using "process == "cron""
Skipping info and debug messages, pass --info and/or --debug to include.
Timestamp          Thread          Type          Activity          PID    TTL
2026-08-05 22:25:45.881118+0530 0x7251c Activity 0x5cf50 36319 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:25:45.883357+0530 0x7251c Activity 0x5cf51 36319 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:25:45.883661+0530 0x7251c Activity 0x5cf52 36319 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:25:45.892700+0530 0x72522 Activity 0x5cf60 36320 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:25:45.893377+0530 0x72522 Activity 0x5cf61 36320 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:25:45.893608+0530 0x72522 Activity 0x5cf62 36320 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:26:01.004439+0530 0x72597 Activity 0x5d020 36330 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:26:01.006315+0530 0x72597 Activity 0x5d021 36330 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:26:01.006837+0530 0x72597 Activity 0x5d022 36330 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:26:01.019798+0530 0x7259d Activity 0x5d030 36331 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:26:01.020963+0530 0x7259d Activity 0x5d031 36331 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:26:01.021282+0530 0x7259d Activity 0x5d032 36331 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:27:00.112867+0530 0x726bc Activity 0x5d1a0 36346 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:27:00.114657+0530 0x726bc Activity 0x5d1a1 36346 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:27:00.115039+0530 0x726bc Activity 0x5d1a2 36346 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:27:00.125043+0530 0x726c3 Activity 0x5d1b0 36347 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:27:00.125944+0530 0x726c3 Activity 0x5d1b1 36347 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:27:00.126333+0530 0x726c3 Activity 0x5d1b2 36347 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:28:00.225040+0530 0x72801 Activity 0x5d2f0 36359 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:28:00.226922+0530 0x72801 Activity 0x5d2f1 36359 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:28:00.227315+0530 0x72801 Activity 0x5d2f2 36359 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:28:00.237262+0530 0x72808 Activity 0x5d300 36360 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:28:00.237923+0530 0x72808 Activity 0x5d301 36360 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:28:00.238223+0530 0x72808 Activity 0x5d302 36360 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:29:00.330225+0530 0x728f7 Activity 0x5d340 36364 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:29:00.332015+0530 0x728f7 Activity 0x5d341 36364 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:29:00.332454+0530 0x728f7 Activity 0x5d342 36364 0 cron: (libsystem_info.dylib) Resolve user group list
2026-08-05 22:29:00.342558+0530 0x728fc Activity 0x5d350 36365 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:29:00.343202+0530 0x728fc Activity 0x5d351 36365 0 cron: (libsystem_info.dylib) Retrieve User by Name
2026-08-05 22:29:00.343403+0530 0x728fc Activity 0x5d352 36365 0 cron: (libsystem_info.dylib) Resolve user group list

Log - Default: 0, Info: 0, Debug: 0, Error: 0, Fault: 0
Activity - Create: 30, Transition: 0, Actions: 0
sivatejamattaparathi@Sivatejas-MacBook-Air backup % crontab -e
crontab: installing new crontab
sivatejamattaparathi@Sivatejas-MacBook-Air backup % cat /Users/sivatejamattaparathi/Desktop/bash_lab/cron_output.log
/bin/bash: /Users/sivatejamattaparathi/Desktop/bash_lab/backup.sh: Operation not permitted
/bin/bash: /Users/sivatejamattaparathi/Desktop/bash_lab/backup.sh: Operation not permitted

```

Conclusion:

This Bash scripting lab helped me attain experience in writing and executing shell scripts to automate common administrative tasks. Throughout the lab, I created scripts for displaying system information, accepting user input, monitoring disk usage, listing files, and automating file backups. I also explored task scheduling using Cron, where I learned how Cron expressions such as `* * * * *` are used to define when a script should run. While testing scheduled backups on macOS Catalina, I investigated a permission issue by reviewing system logs and identified how operating system security restrictions can affect scheduled tasks.

By completing these exercises, I gained practical experience with Bash syntax, variables, loops, conditional statements, file operations, and basic automation techniques. I also developed a better understanding of how scheduled jobs work and how to troubleshoot them when they don't behave as expected. These are practical skills that help reduce repetitive manual work and improve efficiency in Linux and Unix-based environments.